

Arrays

What is an Array

An array is a linear data structure that stores a fixed-size collection of elements of the same type in contiguous memory locations. Because the memory is contiguous, each element can be accessed directly using its index, calculated as $\text{base_address} + (\text{index} * \text{size_of_element})$.

Key Properties

- Fixed size: in most languages, the size must be declared at creation and cannot grow (Python lists are dynamic arrays, an exception).
- Contiguous memory: elements sit next to each other, which is what makes $O(1)$ index access possible.
- Zero-indexed: the first element is at index 0 in most languages including Python, C, C++, and Java.
- Homogeneous: all elements are typically the same data type.

Time Complexity

- Access by index: $O(1)$ — direct calculation, no traversal needed.
- Search (unsorted): $O(n)$ — must check each element in the worst case.
- Search (sorted, using binary search): $O(\log n)$.
- Insertion at end: $O(1)$ amortized (dynamic arrays like Python lists), $O(n)$ for fixed arrays needing resize.
- Insertion/deletion at start or middle: $O(n)$ — all subsequent elements must shift.

Common Operations in Python

```
arr = [10, 20, 30, 40]
```

```
# Access -  $O(1)$ 
```

```
print(arr[2]) # 30
```

```
# Search -  $O(n)$ 
```

```
if 30 in arr:
```

```
    print("found")
```

```
# Insert at end -  $O(1)$  amortized
```

```
arr.append(50)

# Insert at index - O(n), shifts elements
arr.insert(1, 15)

# Delete by value - O(n)
arr.remove(20)

# 2D array (matrix)
matrix = [[1, 2, 3], [4, 5, 6]]
print(matrix[1][2]) # 6
```

When to Use Arrays

Use arrays when you need fast, random access to elements by index and the size of your data is known or changes infrequently. Arrays are the backbone of more complex structures like stacks, queues, hash tables, and are the standard choice for matrices and lookup tables.

Common Interview Problems

- Two Sum — find two numbers that add up to a target (use a hash map for $O(n)$).
- Kadane's Algorithm — maximum subarray sum in $O(n)$.
- Reverse an array in-place using two pointers.
- Rotate an array by k positions.
- Find the missing number in a range using sum formula or XOR.
- Move all zeroes to the end while maintaining order of other elements.